

Git & GitHub

Version control, repositories, commits, branches, collaboration and a basic automation workflow.

Git and GitHub are different

Git is a distributed version-control system. GitHub is a platform that hosts Git repositories and adds collaboration, Pull Requests, Issues, Actions and other services.

Official site and documentation

Use the official Git & GitHub source as the primary reference: <https://git-scm.com/downloads>. Official documentation should be the first source for installation, supported versions, syntax changes and new features.

Installation and first configuration

Install Git from the official site and validate with `git --version`. Configure author name and email with `git config`. In corporate environments, follow identity and authentication policies.

Basic workflow

`git status` shows state; `git add` stages changes; `git commit` records a version; `git push` sends commits; `git pull` integrates remote changes; `git clone` gets a repository for the first time.

Branches

Branches allow isolated development. `git switch -c` creates a branch; `merge` combines histories. Teams may use trunk-based development or other branching strategies.

Pull Request and Code Review

A Pull Request proposes integration and creates space for review, tests and discussion. QA can review impact, added tests, risks and evidence.

Conflicts

Conflicts occur when Git cannot automatically decide how to combine changes. Resolve content consciously, mark it resolved and test before committing.

Automation good practices

Do not version `.env`, secrets, tokens, temporary evidence or caches. Use `.gitignore`, small commits and messages that explain the change.

Git in the QA lifecycle

Versioning automation, synthetic data, documentation and pipelines provides traceability. History helps explain when and why a test changed.

How to start safely

Start from the official Git & GitHub site and documentation. Confirm prerequisites, supported versions, operating system and required permissions. Learn in a controlled environment before using it in a critical workflow.

First QA-oriented use

Pick one small, verifiable task in Git & GitHub, configure only what is necessary, run a simple example and confirm the outcome. Repeat with different data or conditions before adding abstractions.

Using it in a real project

Define where Git & GitHub fits, who owns configuration, how credentials and data are handled and how results are shared. Integrate it into the team workflow instead of creating an isolated activity.

Common issues and troubleshooting

When Git & GitHub fails, verify versions, dependencies, permissions, connectivity, data and environment before assuming a product defect. Preserve the original error and reduce the issue to the smallest reproducible case.